INTEGRATION TESTING

- Data can be lost across an interface; one module can have an inadvertent, adverse affect on another; subfunctions, when combined, may not produce the desired major function; individually acceptable imprecision may be magnified to unacceptable levels; global data structures can present problems.
- Integration testing is a systematic technique for constructing the program structure while at the same time conducting tests to uncover errors associated with interfacing.
- The objective is to take unit tested components and build a program structure that has been dictated by design.

Top-down Integration

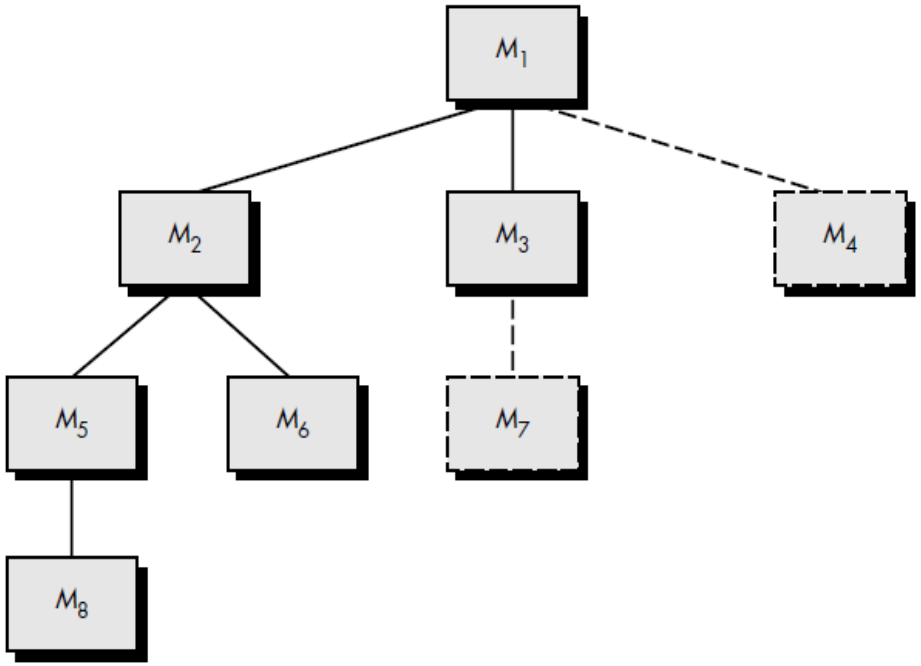
- *Top-down integration testing* is an incremental approach to construction of program structure.
- Modules are integrated by moving downward through the control hierarchy, beginning with the main control module (main program).
- Modules subordinate (and ultimately subordinate) to the main control module are incorporated into the structure in either a depth-first or breadth-first manner.

Depth-first integration would integrate all components on a major control path of the structure. Selection of a major path is somewhat arbitrary and depends on application-specific characteristics.

- Selecting the left hand path, components M1, M2, M5 would be integrated first. Next, M8 or M6. Then, the central and right hand control paths are built.

Breadth-first integration incorporates all components directly subordinate at each level, moving across the structure horizontally.

- Components M2, M3, and M4 (a replacement for stub S4) would be integrated first. The next control level, M5, M6, and so on, follows.



Top down Integration Procedure

- The main control module is used as a test driver and stubs are substituted for all components directly subordinate to the main control module.
- Depending on the integration approach selected (i.e., depth or breadth first), subordinate stubs are replaced one at a time with actual components.
- Tests are conducted as each component is integrated.
- On completion of each set of tests, another stub is replaced with the real component.
- Regression testing may be conducted to ensure that new errors have not been introduced.

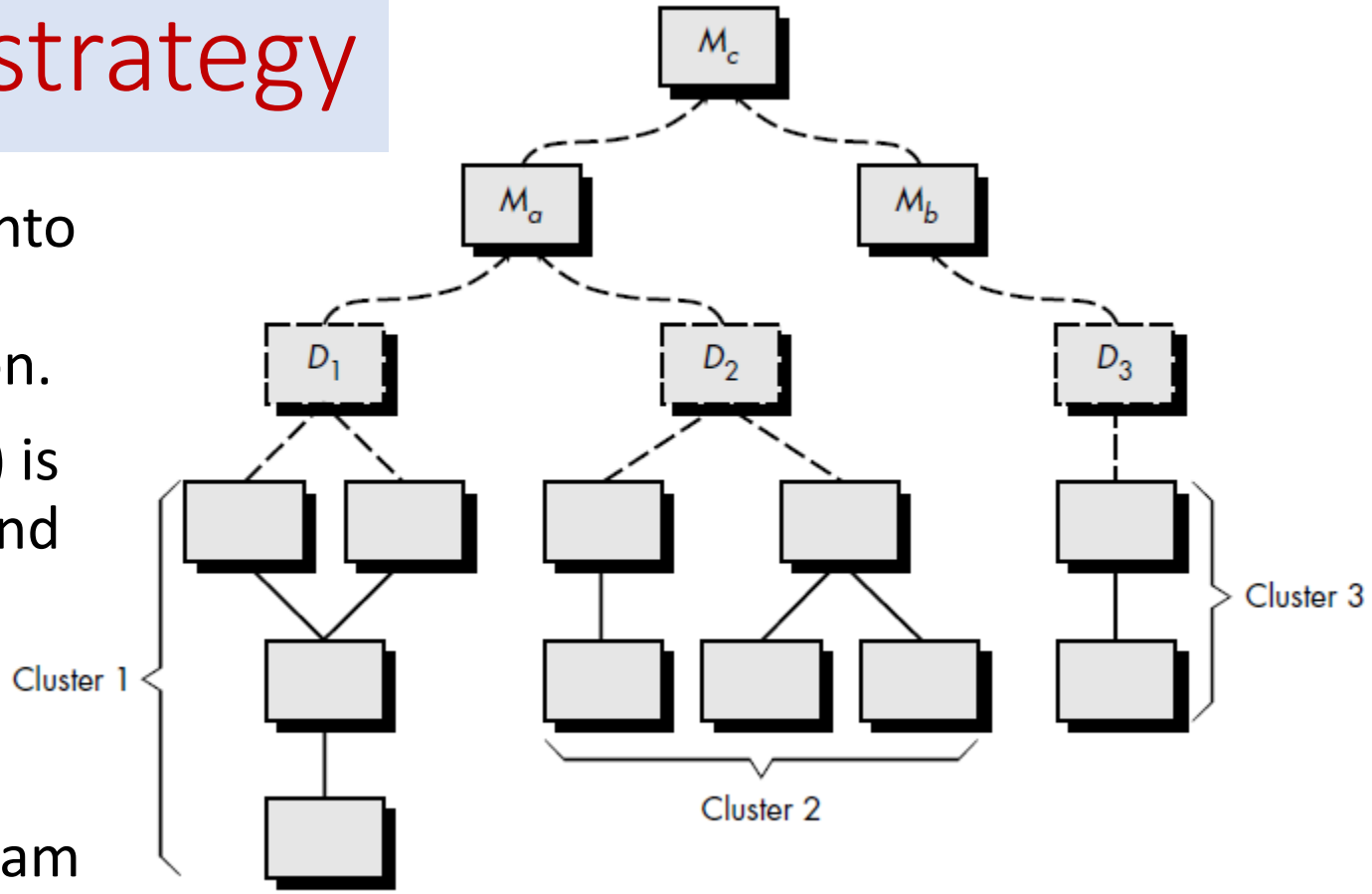
Drawback:
Problems occurs when processing at low levels in the hierarchy is required to adequately test upper levels. Stubs replace low level modules at the beginning of top-down testing; therefore, no significant data can flow upward in the program structure

Bottom-up Integration

- *Bottom-up integration testing*, as its name implies, begins construction and testing with *atomic modules* (i.e., components at the lowest levels in the program structure).
- Because components are integrated from the bottom up, processing required for components subordinate to a given level is always available and the need for stubs is eliminated.

Bottom-up integration strategy

- Low-level components are combined into clusters (sometimes called *builds*) that perform a specific software subfunction.
- A driver (a control program for testing) is written to coordinate test case input and output.
- The cluster is tested.
- Drivers are removed and clusters are combined moving upward in the program structure.



- Components are combined to form clusters 1, 2, and 3.
- Each of the clusters is tested using a driver (shown as a dashed block).
- Components in clusters 1 and 2 are subordinate to M_a.
- Drivers D_1 and D_2 are removed and the clusters are interfaced directly to M_a.
- Similarly, driver D_3 for cluster 3 is removed prior to integration with module M_b.
- Both M_a and M_b will ultimately be integrated with component M_c, and so forth

Regression Testing

- Each time a new module is added as part of integration testing, the software changes.
- New data flow paths are established, new I/O may occur, and new control logic is invoked. These changes may cause problems with functions that previously worked flawlessly.
- In the context of an integration test strategy, *regression testing* is the reexecution of some subset of tests that have already been conducted to ensure that changes have not propagated unintended side effects

The regression test suite (the subset of tests to be executed) contains three different classes of test cases:

- A representative sample of tests that will exercise all software functions.
- Additional tests that focus on software functions that are likely to be affected by the change.
- Tests that focus on the software components that have been changed.

VALIDATION TESTING

- Validation succeeds when software functions in a manner that can be reasonably expected by the customer.
- Software validation is achieved through a series of black-box tests that demonstrate conformity with requirements.
- A test plan outlines the classes of tests to be conducted and a test procedure defines specific test cases that will be used to demonstrate conformity with requirements.
- Both the plan and procedure are designed to ensure that all functional requirements are satisfied, all behavioral characteristics are achieved, all performance requirements are attained, documentation is correct, and human engineered and other requirements are met

Acceptance Testing

Alpha Testing

Beta Testing

Acceptance testing

- When custom software is built for one customer, a series of *acceptance tests* are conducted to enable the customer to validate all requirements.
- Conducted by the enduser rather than software engineers, an acceptance test can range from an informal "test drive" to a planned and systematically executed series of tests.
- In fact, acceptance testing can be conducted over a period of weeks or months, thereby uncovering cumulative errors that might degrade the system over time.

If software is developed as a product to be used by many customers, it is impractical to perform formal acceptance tests with each one. Most software product builders use a process called alpha and beta testing to uncover errors that only the end-user seems able to find.

Alpha Testing:

- The *alpha test* is conducted at the developer's site by a customer. The software is used in a natural setting with the developer "looking over the shoulder" of the user and recording errors and usage problems.
- Alpha tests are conducted in a controlled environment.

Beta Testing:

- The *beta test* is conducted at one or more customer sites by the end-user of the software. Unlike alpha testing, the developer is generally not present.
- Therefore, the beta test is a "live" application of the software in an environment that cannot be controlled by the developer. The customer records all problems (real or imagined) that are encountered during beta testing and reports these to the developer at regular intervals.
- As a result of problems reported during beta tests, software engineers make modifications and then prepare for release of the software product to the entire customer base.

SYSTEM TESTING

- Of the three levels of testing, the system level is closet to everyday experiences.
- System testing is actually a series of different tests whose primary purpose is to fully exercise the computer-based system.
- Although each test has a different purpose, all work to verify that system elements have been properly integrated and perform allocated functions.
- Goal is not to find faults, but to demonstrate performance. Because of this we tend to approach system testing from a functional standpoint rather than from a structural one.
- Since it is so intuitively familiar, system testing in practice tends to be less formal than it might be, and is compounded by the reduced testing interval that usually remains before a delivery deadline.
- During system testing, we should evaluate a number of attributes of the software that are vital to the user by using recovery testing, security testing, stress testing, performance testing.

Recovery Testing

- Many computer based systems must recover from faults and resume processing within a prespecified time. *Recovery testing* is a system test that forces the software to fail in a variety of ways and verifies that recovery is properly performed.
- If recovery is automatic (performed by the system itself), reinitialization, checkpointing mechanisms, data recovery, and restart are evaluated for correctness.
- If recovery requires human intervention, the mean-time-to-repair (MTTR) is evaluated to determine whether it is within acceptable limits.

Security Testing

- Any computer-based system that manages sensitive information or causes actions that can improperly harm (or benefit) individuals is a target for improper or illegal penetration.
- *Security testing* attempts to verify that protection mechanisms built into a system will, in fact, protect it from improper penetration .
- During security testing, the tester plays the role(s) of the individual who desires to penetrate the system.
- The tester may attempt to acquire passwords through external clerical means; may attack the system with custom software designed to breakdown any defenses that have been constructed; may overwhelm the system, thereby denying service to others; may purposely cause system errors, hoping to penetrate during recovery; may browse through insecure data, hoping to find the key to system entry

Stress Testing

- During earlier software testing steps, white-box and black-box techniques resulted in thorough evaluation of normal program functions and performance.
- Stress tests are designed to confront programs with abnormal situations.
- In essence, the tester who performs stress testing asks: "how high can we crank this up before it fails?“
- *Stress testing* executes a system in a manner that demands resources in abnormal quantity, frequency, or volume. Examples:
  1. Special tests may be designed that generate ten interrupts per second, when one or two is the average rate
  2. Input data rates may be increased by an order of magnitude to determine how input functions will respond
  3. Test cases that require maximum memory or other resources are executed,

Performance Testing

- For real-time and embedded systems, software that provides required function but does not conform to performance requirements is unacceptable.
- *Performance testing* is designed to test the run-time performance of software within the context of an integrated system.
- Performance testing occurs throughout all steps in the testing process. Even at the unit level, the performance of an individual module may be assessed as white-box tests are conducted.
- However, it is not until all system elements are fully integrated that the true performance of a system can be ascertained